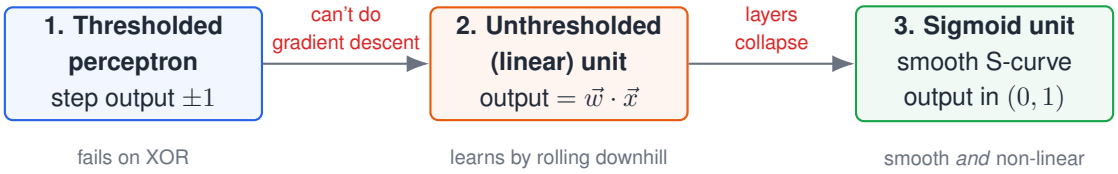
From the Perceptron to the Sigmoid Unit

Explained simply · with intuition, maths, worked examples and the repo's own code

Folder: `01 - Perceptron and Gradient Descent` — Code: `01_thresholded_perceptron.py`, `02_unthresholded_delta_rule.py`, `03_sigmoid_unit_non_linearity.py`

What this guide is about. We build one artificial neuron, see where it breaks, and fix it twice. Each fix is a tiny change to the maths and a tiny change to the code. By the end you will know *why* modern networks use smooth activation functions like the sigmoid, and you will be ready for backpropagation (the next folder).



1 Why copy the brain at all?

The handwritten course notes start with a few facts about the brain:

Fact	Number	What it tells us
Neurons	$\approx 10^{11}$	huge number of *very simple* units
Connections per neuron	$\approx 10^{4}$	each unit listens to many others
Switching time of one neuron	$\approx 10^{-3}$ s	each unit is *slow*…
Time to recognise a face	$\approx 10^{-1}$ s	…so only ∼100 steps can happen in a row

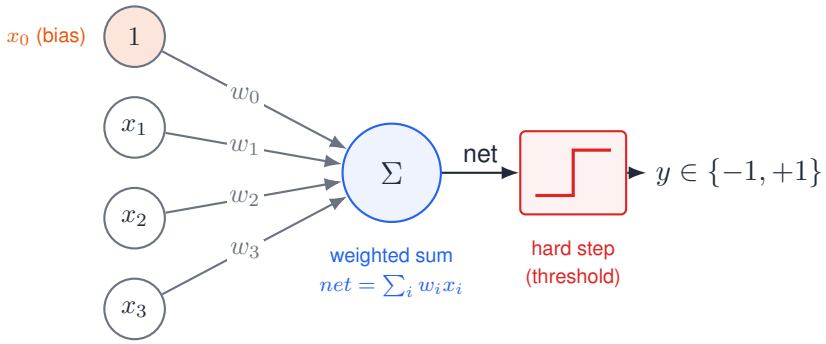
Intuition

Only about 100 slow steps, yet the brain recognises a face. So it cannot be running a long step-by-step program. It must be doing **lots of simple work in parallel**, with knowledge spread over many connection strengths (*distributed representation*). Nobody "coded the algorithm" – the connections were *learned*. Artificial neural networks copy exactly this idea: many simple units + learnable connection strengths (weights).

2 The Thresholded Perceptron

2.1 What one neuron computes

A perceptron takes some inputs $x_1, x_2, \ldots, x_n$, multiplies each by a **weight** w_i (how important that input is), adds everything up, and then makes a yes/no decision.



$$net = \sum_{i=0}^{n} w_i x_i = w_0 + w_1 x_1 + \cdots + w_n x_n \qquad y = \begin{cases} +1 & \text{if } net > 0 \\ -1 & \text{otherwise} \end{cases}$$

Everyday example: "Should I go to the cricket match?"

Inputs: $x_1 =$ is it a weekend? (1/0), $x_2 =$ is a friend coming? (1/0), $x_3 =$ is it raining? (1/0).
Weights you care about: $w_1 = +2$, $w_2 = +1$, $w_3 = -3$, and a bias $w_0 = -1.5$ (you are a bit lazy by default).
Saturday, friend coming, no rain: $net = -1.5 + 2(1) + 1(1) - 3(0) = 1.5 > 0 \Rightarrow y = +1$ (go!)
Saturday, friend coming, raining: $net = -1.5 + 2 + 1 - 3 = -1.5 \leq 0 \Rightarrow y = -1$ (stay home)
A big positive weight means "this input pushes me towards YES"; a negative weight pushes towards NO.

2.2 Why the extra input $x_0 = 1$ (the bias)?

The handwritten notes say: *"do not want output to be 0 when all inputs are 0"*. Without w_0, if every $x_i = 0$ then $net = 0$ no matter what the weights are – the neuron has no opinion of its own. Adding a fake input $x_0 = 1$ with its own weight w_0 gives the neuron a **default tendency**. Geometrically, it lets the decision line move away from the origin.

In the code

```
X_bias = np.c_[np.ones((X.shape[0], 1)), X]   # glue a column of 1s in front: x0 = 1
```

`np.c_` sticks columns side by side. Input $[0, 1]$ becomes $[1, 0, 1]$, so w_0 (the first weight) is the bias.

2.3 The picture: a straight line that splits the plane

With two inputs, the neuron says YES when $w_0 + w_1 x_1 + w_2 x_2 > 0$. The border where it switches,

$$w_0 + w_1 x_1 + w_2 x_2 = 0,$$

is the equation of a **straight line**. Everything on one side is $+1$, the other side is -1. In 3-D it is a flat plane, and in many dimensions we call it a **hyperplane**. So:

Big idea

A perceptron = "draw one straight line, and say YES on one side". Learning = moving and rotating that line until the YES points and NO points are on the correct sides.

2.4 How it learns: the perceptron rule

Show the perceptron one example $(\vec{x}, t)$ where t is the true answer (target) and o is what the neuron said. Then nudge every weight:

$$w_i \leftarrow w_i + \Delta w_i, \qquad \Delta w_i = \eta\,(t - o)\,x_i$$

η ("eta") is the **learning rate**: how big a nudge we take (e.g. 0.1). Let us read the rule case by case:

Target t	Output o	$t - o$	What happens
$+1$	$+1$	0	Correct $\Rightarrow$ no change.
-1	-1	0	Correct $\Rightarrow$ no change.
$+1$	-1	$+2$	Said NO, should be YES $\Rightarrow$ **add** $2\eta x_i$: weights of active inputs go up, so net goes up next time.
-1	$+1$	-2	Said YES, should be NO $\Rightarrow$ **subtract** $2\eta x_i$: net goes down next time.

Intuition

Only inputs that were *switched on* ($x_i \neq 0$) get blamed. If $x_i = 0$ that input could not have caused the mistake, so $\Delta w_i = \eta(t - o) \cdot 0 = 0$. It is like a teacher who corrects only the students who actually spoke.

2.5 Worked example: learning the AND gate (exact numbers from the code)

AND is YES only for $(1, 1)$. Targets use ± 1: $(0,0) \to -1,\ (0,1) \to -1,\ (1,0) \to -1,\ (1,1) \to +1$. Start with $\vec{w} = (w_0, w_1, w_2) = (0, 0, 0)$, $\eta = 0.1$.

Epoch 1, step by step

input (x_1, x_2)	t	$net = \vec{w} \cdot (1, x_1, x_2)$	o	$\Delta\vec{w} = 0.1(t - o)(1, x_1, x_2)$	new $\vec{w}$
$(0,0)$	-1	0	-1	$(0,0,0)$	$(0,0,0)$
$(0,1)$	-1	0	-1	$(0,0,0)$	$(0,0,0)$
$(1,0)$	-1	0	-1	$(0,0,0)$	$(0,0,0)$
$(1,1)$	$+1$	0	-1	$0.1 \cdot 2 \cdot (1,1,1) = (0.2, 0.2, 0.2)$	$(0.2, 0.2, 0.2)$

Note: $net = 0$ is *not* > 0, so the output is -1. Only the last example was wrong, so only it caused an update.

Running more epochs (the code does this), the mistakes per epoch go $1, 3, 3, 2, 1, 0$. In epoch 6 there are no mistakes, so the weights stay at

$$\vec{w} = (-0.4,\ 0.4,\ 0.2) \quad \Longrightarrow \quad net = -0.4 + 0.4x_1 + 0.2x_2.$$

Check: $(0,0) : -0.4,\ (0,1) : -0.2,\ (1,0) : 0,\ (1,1) : +0.2$. Only $(1,1)$ is > 0 – AND is learned.

In the code: `01_thresholded_perceptron.py`

```python
class ThresholdedPerceptron:
    def __init__(self, learning_rate=0.1, epochs=10):
        self.eta = learning_rate          # size of each nudge
        self.epochs = epochs              # how many passes over the data
        self.w = None

    def fit(self, X, y):
        X_bias = np.c_[np.ones((X.shape[0], 1)), X]    # add x0 = 1
        self.w = np.zeros(X_bias.shape[1])             # start: all weights 0
        for _ in range(self.epochs):                   # repeat the whole dataset
            for xi, target in zip(X_bias, y):          # one example at a time
                net = np.dot(xi, self.w)               # weighted sum
                output = 1 if net > 0 else -1          # the hard step
                delta_w = self.eta * (target - output) * xi   # perceptron rule
                self.w += delta_w                      # nudge

    def predict(self, X):
        X_bias = np.c_[np.ones((X.shape[0], 1)), X]
        return np.where(np.dot(X_bias, self.w) > 0, 1, -1)
```

Line by line. Line 8 adds the bias column. Line 9 starts from "no opinion". Lines 11–15 are exactly the maths above: weighted sum → step → rule $\Delta w_i = \eta(t-o)x_i$ (the whole vector `xi` is updated at once). `predict` does the same forward step for many points at once; `np.where` turns "> 0" into $+1$, else -1.

2.6 Where it breaks: XOR

The script trains the same class on AND, OR and XOR and colours the plane by the prediction:

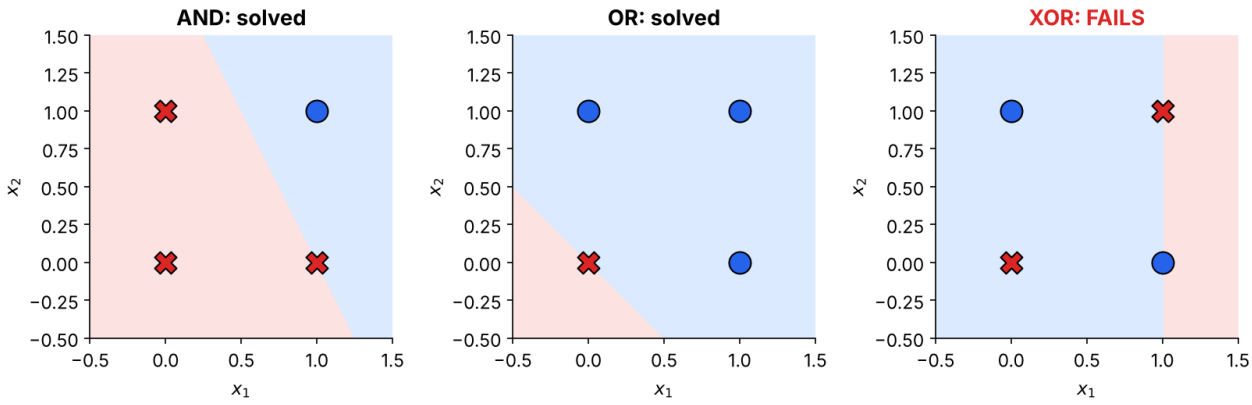


Figure 1: Blue circles should be $+1$, red crosses -1. AND and OR can be split by one line. XOR cannot – the perceptron's best attempt still gets points wrong.

Why is XOR impossible? A 4-line proof. XOR wants $(0,1)$ and $(1,0)$ to be YES and $(0,0)$, $(1,1)$ to be NO:

$$
\begin{aligned}
(0,0) \to \text{NO}: &\quad w_0 \le 0 \\
(1,1) \to \text{NO}: &\quad w_0 + w_1 + w_2 \le 0 \\
(0,1) \to \text{YES}: &\quad w_0 + w_2 > 0 \\
(1,0) \to \text{YES}: &\quad w_0 + w_1 > 0
\end{aligned}
$$

Add the two YES lines: $2w_0 + w_1 + w_2 > 0$. Add the two NO lines: $2w_0 + w_1 + w_2 \leq 0$. The same quantity cannot be both > 0 and ≤ 0. **Contradiction** – no weights exist. One straight line can never do XOR.

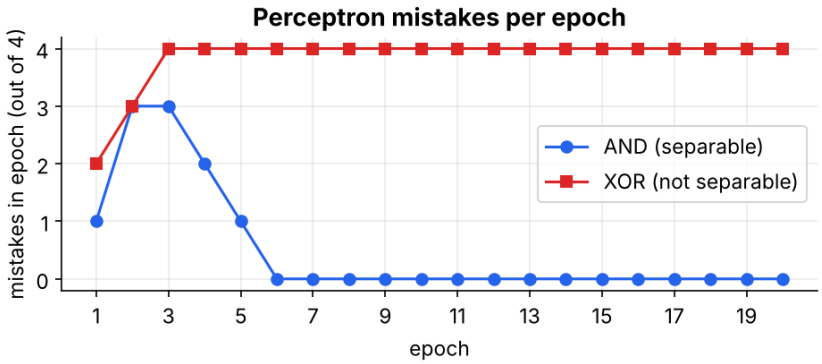


Figure 2: On AND the mistakes drop to 0 and learning stops. On XOR the weights go round in a circle forever: every epoch ends at the same $\vec{w} = (0.2, -0.2, 0)$ with all 4 examples wrong at the moment they are seen.

Two problems we must fix

1. **One line is not enough** (XOR). We will need *several* neurons in layers.
2. **The step function blocks learning in layers.** Its slope is 0 everywhere (and undefined at 0). If a hidden neuron's output only jumps between -1 and $+1$, a tiny change in its weights usually changes *nothing*, so we can't tell which way to move it. The notes say: *"Do not know how to update intermediate neurons"* and *"Get rid of thresholding to make output differentiable w.r.t. input"*.

3 Unthresholded Perceptron and the Delta Rule

3.1 Step 1: just remove the step

The simplest fix: don't threshold. The output is the raw weighted sum,

$$o(\vec{x}) = \vec{w} \cdot \vec{x} = w_0 + w_1 x_1 + \cdots + w_n x_n$$

Now the output is a real number like 3.7, not just ± 1. That lets us measure *how wrong* we are, not only *whether* we are wrong.

3.2 Step 2: one number that says "how bad" – the cost function

For a dataset D of examples d with targets t_d and outputs o_d:

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

- **Why square?** So that being too high and too low both count as bad (no cancelling), and big mistakes count much more than small ones.
- **Why $\frac{1}{2}$?** Pure convenience: when we differentiate $(\cdot)^2$ a 2 comes down, and the $\frac{1}{2}$ cancels it.
- E depends only on the weights (the data is fixed). Different $\vec{w} \Rightarrow$ different error. Learning = find the $\vec{w}$ with the smallest E.

3.3 Step 3: roll downhill – gradient descent

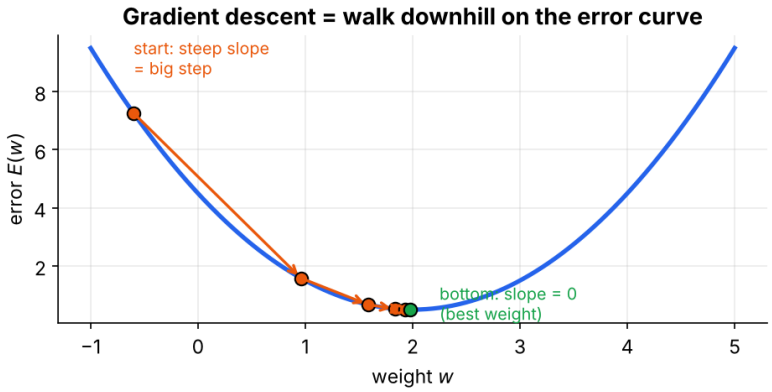


Figure 3: For a linear unit, E is a smooth bowl. The slope tells us which way is uphill; we step the other way. Steps get smaller near the bottom because the slope gets smaller.

Intuition: walking down a hill in fog

You can't see the valley, but you can feel the slope under your feet. Take a small step in the steepest downhill direction, feel again, repeat. The **gradient** ∇E is that "slope under your feet" – it points uphill, so we move *against* it:

$$\vec{w} \leftarrow \vec{w} - \eta \nabla E, \qquad \nabla E = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n}\right].$$

Each $\frac{\partial E}{\partial w_i}$ answers: "if I increase w_i a tiny bit, does the error go up or down, and how fast?"

3.4 Step 4: compute the gradient (slowly, one line at a time)

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_d (t_d - o_d)^2 && \text{(definition)} \\
&= \frac{1}{2} \sum_d 2\,(t_d - o_d) \cdot \frac{\partial}{\partial w_i}(t_d - o_d) && \text{(chain rule: derivative of } u^2 \text{ is } 2u\,u') \\
&= \sum_d (t_d - o_d) \cdot \left(-x_{i,d}\right) && (t_d \text{ is fixed; } o_d = \textstyle\sum_j w_j x_{j,d} \text{ so } \partial o_d/\partial w_i = x_{i,d}) \\
&= -\sum_d (t_d - o_d)\, x_{i,d}.
\end{aligned}$$

Move against it ($\Delta w_i = -\eta\, \partial E/\partial w_i$) and you get the **Delta rule** (also called LMS or Widrow–Hoff):

$$\boxed{\Delta w_i = \eta \sum_{d \in D} (t_d - o_d)\, x_{i,d}}$$

Look familiar? It is the perceptron rule again! The difference: o is now a real number, so $(t - o)$ is a *size* of error (e.g. 0.4), not just 0 or ± 2. Small errors give small updates, big errors give big ones.

One gradient step by hand

One data point: $x = 2$, target $t = 7$. Start with $w_0 = 0,\ w_1 = 1$, $\eta = 0.05$.

- Output: $o = w_0 + w_1 x = 0 + 1 \cdot 2 = 2$. Error: $t - o = 7 - 2 = 5$. Cost $E = \frac{1}{2} \cdot 5^2 = 12.5$.

- Updates: $\Delta w_0 = \eta(t-o)\cdot 1 = 0.05\cdot 5\cdot 1 = 0.25; \quad \Delta w_1 = \eta(t-o)\cdot x = 0.05\cdot 5\cdot 2 = 0.5.$
- New weights: $w_0 = 0.25,\ w_1 = 1.5$. New output: $0.25 + 1.5\cdot 2 = 3.25$. New cost $\frac{1}{2}(7-3.25)^2 = 7.03$.

The error dropped from 12.5 to 7.03 in one step. Note w_1 moved more than w_0 because its input ($x = 2$) was bigger – bigger input, bigger share of the blame.

3.5 Batch vs stochastic gradient descent

- **Batch GD:** look at *all* examples, add up their gradients, then take *one* careful step. Smooth but each step is expensive.
- **Stochastic (incremental) GD:** take a step after *every single* example (in shuffled order). Each step is cheap and a bit noisy, but you take N steps per epoch.

Analogy

Batch = asking the whole class before changing your answer. Stochastic = adjusting a little after each student speaks. The second is noisier but usually gets close much faster.

In the code: `02_unthresholded_delta_rule.py`

```
if self.mode == 'batch':
    outputs  = np.dot(X_bias, self.w)        # o_d for ALL examples at once
    errors   = y - outputs                   # (t_d - o_d) for every d
    cost     = 0.5 * np.sum(errors ** 2)     # E(w)
    gradient = -np.dot(X_bias.T, errors)     # dE/dw_i = -sum_d (t_d - o_d) x_id
    self.w  -= self.eta * gradient           # one step downhill

elif self.mode == 'stochastic':
    for i in np.random.permutation(N):       # shuffled order
        output   = np.dot(X_bias[i], self.w) # one example
        error    = y[i] - output
        gradient = -error * X_bias[i]        # gradient from this example only
        self.w  -= self.eta * gradient       # step immediately
```

Matching code to maths. `X_bias.T @ errors` computes $\sum_d (t_d - o_d)x_{i,d}$ for every i in one go: row i of `X_bias.T` holds input i for all examples, and the dot product with `errors` is exactly that sum. The minus sign and `-=` together give $+\eta\sum(t-o)x$, the Delta rule.
Note: this file *sums* over the 100 examples (no division by N), so the gradient is about 100$\times$ bigger. That is why the batch learning rate is tiny (10^{-4}). Dividing by N (a mean) would allow a bigger η; both are fine.

The experiment: 100 points on the line $t = 3x + 5$ plus noise. Both learners start at $\vec{w} = (0, 0)$ and run 100 epochs.

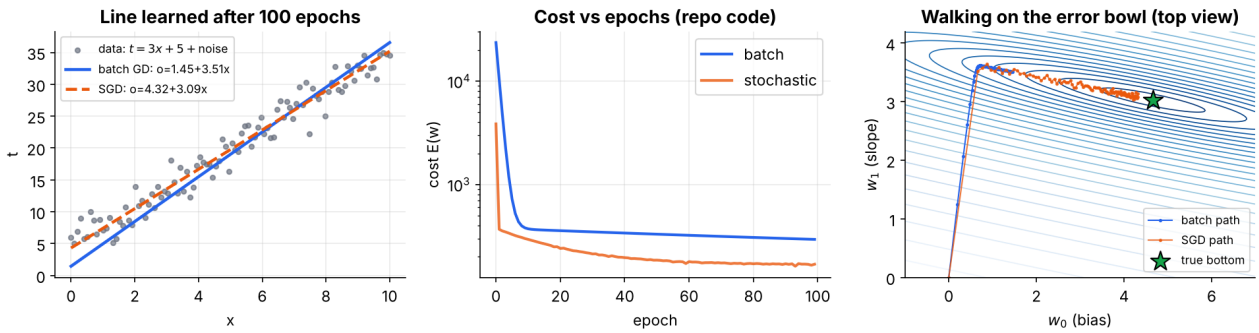


Figure 4: Left: SGD ends at $o = 4.32 + 3.09x$, close to the best possible line $4.65 + 3.03x$. Batch GD gets the slope (3.51) but its bias is still only 1.45. Middle: cost per epoch (log scale). Right: the paths on a top-down map of the error bowl.

Why is batch GD so slow on the bias?

The right panel shows the bowl is a long, narrow *valley*: the error is very sensitive to the slope w_1 (because x goes up to 10) but not very sensitive to the bias w_0. With one small learning rate for both, the walker quickly falls into the valley floor and then crawls along it. This "narrow valley" problem is why people later invented feature scaling, momentum and Adam.

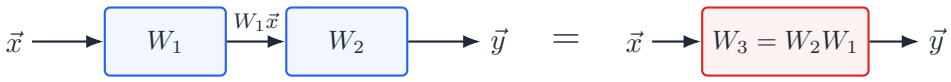
Choosing the learning rate η

Too small $\Rightarrow$ tiny steps, painfully slow. Too big $\Rightarrow$ you jump over the bottom of the bowl, land higher on the other side and the error can explode. Always plot the cost curve: it should go down smoothly.

4 The Sigmoid Unit and Non-linearity

4.1 Problem: stacking linear units gives... one linear unit

We wanted layers of neurons to solve XOR. But if every neuron is linear, layers are useless:



$$\vec{y} = W_2(W_1\vec{x}) = (W_2W_1)\,\vec{x} = W_3\,\vec{x}.$$

Tiny example with single numbers: layer 1 doubles ($\times 2$), layer 2 triples ($\times 3$). Together: $\times 6$ – one layer could have done that. Ten linear layers still give one straight line. The notes say: *"Network of linear neurons will have the same effect as a single neuron."*

In the code: the collapse, proved numerically

```
X  = np.random.randn(10, 3)                  # 10 samples, 3 features
W1 = np.random.randn(3, 4)                   # layer 1: 3 -> 4
W2 = np.random.randn(4, 2)                   # layer 2: 4 -> 2
output_stacked   = np.dot(np.dot(X, W1), W2)   # X -> layer1 -> layer2
W_collapsed      = np.dot(W1, W2)              # one 3 -> 2 matrix
output_collapsed = np.dot(X, W_collapsed)
np.allclose(output_stacked, output_collapsed)  # prints True
```

(The code uses rows = samples, so it writes XW_1W_2 instead of $W_2W_1\vec{x}$. Same idea.)

4.2 What we need: a smooth version of the step

From the handwritten notes, the new activation must be:

1. **non-linear** – so layers don't collapse;
2. **continuous and differentiable** – so gradient descent can feel a slope;
3. **step-like** – low for very negative input, high for very positive input, like a neuron firing.

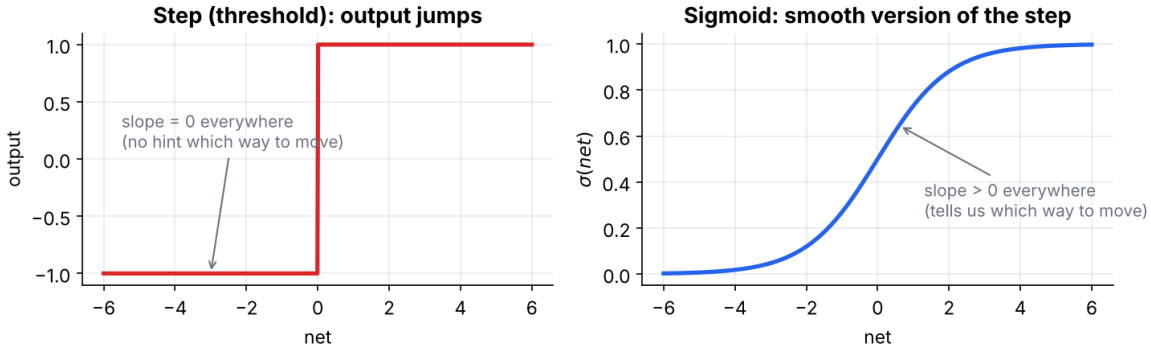


Figure 5: The step is flat everywhere (slope 0) so gradient descent is blind. The sigmoid has the same overall shape but bends smoothly, so every point has a useful slope.

4.3 The sigmoid function

$$\boxed{\sigma(x) = \frac{1}{1 + e^{-x}}}$$

x	-5	-2	-1	0	1	2	5
e^{-x}	148.4	7.39	2.72	1	0.37	0.135	0.0067
$\sigma(x)$	0.007	0.119	0.269	**0.5**	0.731	0.881	0.993

Read it as a "soft yes/no": very negative $\to$ almost 0 (NO), very positive $\to$ almost 1 (YES), and 0 $\to$ exactly 0.5 (unsure). The output can be read like a probability.

4.4 Its derivative is beautifully simple: $y(1 - y)$

Let $y = \sigma(x) = (1 + e^{-x})^{-1}$.

$$\begin{aligned}
\frac{dy}{dx} &= -1 \cdot (1 + e^{-x})^{-2} \cdot \frac{d}{dx}(1 + e^{-x}) && \text{(chain rule on } u^{-1}\text{)} \\
&= -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2} \\
&= \underbrace{\frac{1}{1 + e^{-x}}}_{y} \cdot \underbrace{\frac{e^{-x}}{1 + e^{-x}}}_{1-y} && \text{(because } 1 - y = \tfrac{1+e^{-x}-1}{1+e^{-x}} = \tfrac{e^{-x}}{1+e^{-x}}\text{)}
\end{aligned}$$

$$\Rightarrow \quad \frac{d\sigma}{dx} = \boxed{y\,(1 - y)}$$

Why this is great: we already computed y in the forward pass, so the slope costs just one multiply – no extra exponentials. Example: at $x = 0$, $y = 0.5$, slope $= 0.5 \times 0.5 = 0.25$ (the maximum). At $x = 5$, $y = 0.993$, slope $= 0.993 \times 0.007 \approx 0.007$ (almost flat).

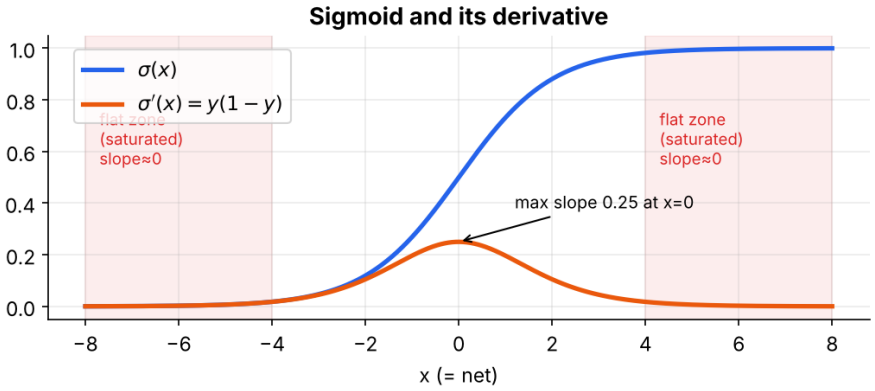


Figure 6: The slope is at most 0.25 and almost 0 in the flat "saturated" zones. Remember this: it causes *vanishing gradients* in deep networks (next folder).

4.5 Training one sigmoid neuron

Now $o = \sigma(net)$ with $net = \vec{w} \cdot \vec{x}$. Same squared error. The chain rule has three links – error, then the sigmoid, then the weighted sum:

$$\frac{\partial E}{\partial w_i} = \underbrace{\frac{\partial E}{\partial o}}_{-(t-o)} \cdot \underbrace{\frac{\partial o}{\partial net}}_{o(1-o)} \cdot \underbrace{\frac{\partial net}{\partial w_i}}_{x_i} = -(t-o)\, o(1-o)\, x_i.$$

Read it as "blame = error × sensitivity × input"

- $(t-o)$ – **how wrong** were we?
- $o(1-o)$ – **how sensitive** is the neuron right now? Near 0.5 it reacts strongly; saturated near 0 or 1 it barely reacts.
- x_i – **how much did this input contribute**?

Compared with the linear unit, only the middle factor is new. The handwritten notes say exactly this: *"if the activation function changes, only this partial derivative changes."*

In the code: 03_sigmoid_unit_non_linearity.py

```python
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(output):
    return output * (1 - output)              # y(1 - y): uses the OUTPUT, not x

class SigmoidNeuron:
    def fit(self, X, y):
        X_bias = np.c_[np.ones((X.shape[0], 1)), X]
        self.w = np.random.randn(X_bias.shape[1])        # small random start
        for _ in range(self.epochs):
            net    = np.dot(X_bias, self.w)              # forward: weighted sum
            output = sigmoid(net)                        # forward: squash
            error  = y - output                          # (t - o)
            self.loss_history.append(np.mean(error**2))
            delta    = error * sigmoid_derivative(output)        # (t-o) * o(1-o)
            gradient = -np.dot(X_bias.T, delta) / X.shape[0]     # mean over examples
            self.w  -= self.eta * gradient                       # step downhill
```

`delta` is the "error $\times$ sensitivity" part for every example. `X_bias.T @ delta` multiplies by the input and sums over examples – the same trick as in the Delta rule. Here the gradient is divided by N (a mean), so a large $\eta = 0.5$ is fine.

Result on the OR gate (targets 0/1), 500 epochs, $\eta = 0.5$

Final weights $\vec{w} \approx (-0.91,\ 2.54,\ 2.53)$. Outputs:

input	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$
target	0	1	1	1
output $\sigma(net)$	0.287	0.835	0.837	0.985

Check one by hand: $(0,1)$: $net = -0.91 + 2.53 = 1.62$, $\sigma(1.62) = 1/(1+e^{-1.62}) = 1/(1+0.198) = 0.835$. ✓

All are on the correct side of 0.5. The loss fell from 0.351 to 0.034. More epochs would push them closer to 0 and 1.

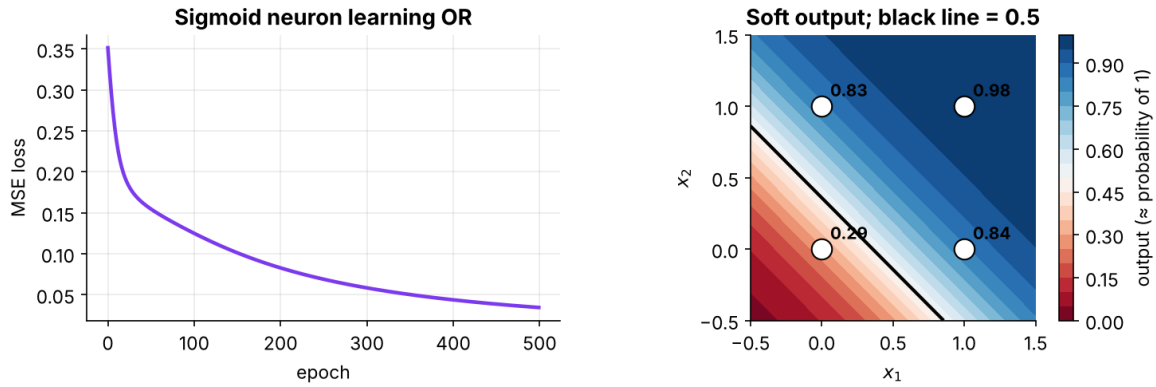
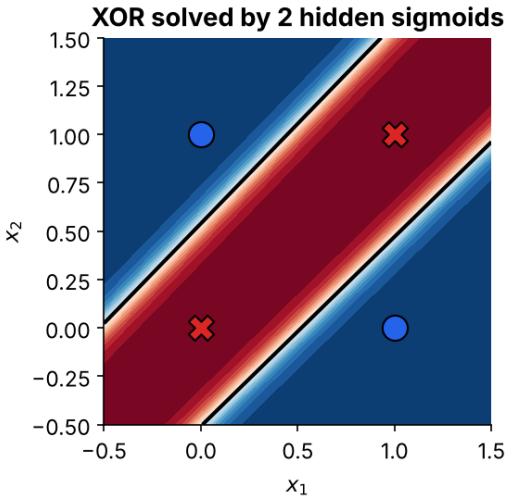
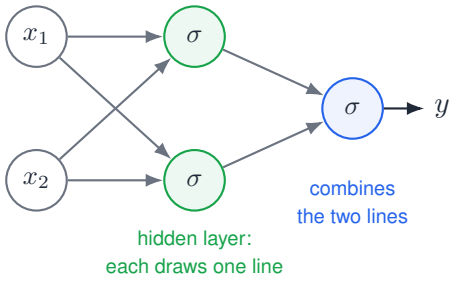


Figure 7: Left: the loss falls smoothly – no more jumping around. Right: the sigmoid gives a soft, graded answer; the black 0.5 line is still a straight line.

Important: one sigmoid neuron still cannot do XOR

The decision border $\sigma(net) = 0.5$ means $net = 0$, which is still a straight line. The sigmoid only becomes powerful when we put several of them in **layers**: then the non-linearity stops the layers collapsing, and their lines combine into curved borders. A network with just 2 hidden sigmoids solves XOR:





Trained outputs: $(0,0) \to 0.01$, $(0,1) \to 0.99$, $(1,0) \to 0.99$, $(1,1) \to 0.01$. Each hidden neuron draws one line; the output neuron combines them into a band. But how do we train the *hidden* weights? That is **backpropagation** – the next guide.

5 Everything on one page

	Thresholded perceptron	Unthresholded (linear) unit	Sigmoid unit
Output	step$(\vec{w} \cdot \vec{x}) \in \{-1, +1\}$	$\vec{w} \cdot \vec{x}$ (any real)	$\sigma(\vec{w} \cdot \vec{x}) \in (0, 1)$
Error measured as	right / wrong	$E = \frac{1}{2}\sum(t-o)^2$	$E = \frac{1}{2}\sum(t-o)^2$ (or mean)
Update Δw_i	$\eta(t-o)x_i$	$\eta \sum_d (t_d - o_d) x_{i,d}$	$\eta \sum_d (t_d - o_d)\, o_d(1 - o_d)\, x_{i,d}$
Differentiable?	No (slope 0)	Yes	Yes
Stacking layers helps?	Can't be trained	No – collapses to 1 layer	Yes – non-linear
XOR with one unit?	No	No	No (needs a hidden layer)
Repo file	`01_thresholded_perceptron.py`	`02_unthresholded_delta_rule.py`	`03_sigmoid_unit_non_linearity.py`

Key formulas to remember

$$net = \sum_i w_i x_i \qquad E = \tfrac{1}{2}\sum_d (t_d - o_d)^2 \qquad \vec{w} \leftarrow \vec{w} - \eta \nabla E \qquad \sigma(x) = \frac{1}{1+e^{-x}} \qquad \sigma' = y(1-y)$$

Check yourself (answers below)

1. With $\vec{w} = (-0.4, 0.4, 0.2)$, what does the perceptron output for input $(1, 0)$, and why?
2. In the Delta rule, why does an input with $x_i = 0$ never change its weight?
3. What is $\sigma'(x)$ when the neuron outputs $y = 0.9$?

4. Why can't ten stacked linear layers solve XOR?
5. A sigmoid neuron outputs 0.999 but the target is 0. Why does it learn slowly (with squared error)?

Answers. 1) $net = -0.4 + 0.4 = 0$, not > 0, so -1. 2) $\Delta w_i \propto x_i = 0$: that input played no part in the output. 3) $0.9 \times 0.1 = 0.09$. 4) They multiply out to one matrix – still one straight line. 5) The factor $o(1-o) = 0.999 \times 0.001 \approx 0.001$ makes the gradient tiny even though the error is huge (the next guide shows cross-entropy fixes this).